1. Análisis y definición del escenario

El proyecto establece una infraestructura descentralizada para la gestión de campañas de fidelización, sorteos y canje de premios promocionales entre múltiples entidades independientes y potencialmente desconfiadas entre sí.

En los sistemas tradicionales, las promociones comerciales (como los sorteos en supermercados) suelen generar dudas sobre la autenticidad de los ganadores o la manipulación de los códigos premiados. Esta plataforma resuelve ese problema permitiendo que:

1. Las **Marcas Proveedoras** depositen directamente las recompensas (fondos/tokens) en un contrato de custodia (*Escrow*) sin pasar por las cuentas del comercio distribuidor.
2. Un **Auditor Notarial** registre inmutablemente los *hashes* criptográficos de los códigos premiados antes de iniciar la campaña.
3. Los **Clientes** canjeen códigos físicos o digitales desde su *wallet*, recibiendo premios automáticos de forma transparente y acumulando reputación (*Tiers*).

1.2 Identificación y Roles de los Actores (Interlocutores)

Para dotar al sistema de la complejidad exigida, se definen **4 actores principales** con permisos y responsabilidades bien delimitadas:

1. Comercio Distribuidor (Ej. Supermercado / Gadis):

- Emite los códigos promocionales a los clientes tras las compras.
- Configura las reglas generales de la campaña (fechas de inicio y fin, límite de canjes por usuario).

2. Marcas Proveedoras / Patrocinadores (Ej. Coca-Cola, Nestlé):

- Financian y respaldan premios específicos vinculados a sus productos.
- Inyectan fondos en las cuentas de custodia en *Escrow* del Smart Contract.

3. Auditor / Notario Independiente:

- Entidad externa encargada de validar las bases legales de la promoción.
- Firma mediante clave criptográfica o multifirma (*Multisig*) el compromiso de *hashes* (keccak256) de los boletos premiados antes del arranque público.

4. Cliente / Usuario Final:

- Interactúa directamente con la blockchain conectando su *wallet* (MetaMask).
- Ejecuta la verificación de sus códigos promocionales.

- Recibe las transferencias automáticas de premios y sube de nivel de fidelización (*Tier*) en la red.

1.3 Justificación del Uso de una Blockchain Pública (Ethereum / Sepolia)

El enunciado exige justificar expresamente por qué este escenario requiere las propiedades de una **blockchain pública** frente a una solución de software tradicional:

Base de Datos Tradicional	Smart Contract en Ethereum
Centralizada: Administrada por el comercio. Modificable: Los premios se pueden alterar. Opaca: Sorteos y entregas no auditables. Custodia manual de los fondos de marcas.	Descentralizada: Ejecución en nodos neutrales. Inmutable: Esquema Commitment Scheme. Transparente: Verificable en Etherscan. Custodia automatizada en Escrow.

No-Repudio y Esquema de Compromiso (*Commitment Scheme*):

Antes de que comience la campaña, el notario y el supermercado registran el *hash* de cada código premiado. Gracias a la inmutabilidad de Ethereum, resulta matemáticamente imposible que el supermercado altere la lista de ganadores o simule un reparto fraudulento a posteriori.

Automatización de Custodia (*Escrow*) sin Intermediarios:

Las marcas no tienen que fiarse del supermercado para entregar el dinero de los premios. La blockchain actúa como un custodio neutral: cuando el contrato valida un código ganador, **libera y transfiere el fondo directamente al cliente**, eliminando el riesgo de impago o apropiación indebida.

Auditoría Universal Pública:

Al desplegarse en una red pública (como la Testnet Sepolia), cualquier participante, organismo regulador o consumidor puede auditar la lógica del contrato, verificar la equidad del reparto y rastrear cada transacción en tiempo real a través de un explorador de bloques como Etherscan.

Resistencia a la Manipulación e Interrupción:

Dado que no existe un servidor central (SPOF - *Single Point of Failure*), el sistema no puede ser apagado ni manipulado unilateralmente por ninguna de las partes involucradas.

Conceptos Clave

¿Qué es *Escrow*?

Es una cuenta de custodia o depósito en garantía. La marca (ej. Coca-Cola) deposita el dinero del premio directamente en el Smart Contract. El contrato retiene ese dinero de forma neutral y solo lo libera automáticamente al cliente cuando introduce un código ganador. Ni el supermercado ni la marca pueden tocar ese dinero mientras dure la promoción.

¿Qué es *Loyalty*?

Significa fidelización. Es el sistema que premia a los clientes recurrentes.

¿Qué es *Tier*?

Es la categoría o nivel de fidelidad del cliente (por ejemplo: Nivel 0 = Bronce, Nivel 1 = Plata, Nivel 2 = Oro). A medida que el cliente canjea códigos, gana puntos y sube de *Tier*, lo que le permite acceder a premios exclusivos o de mayor valor.

¿Qué es *Commitment Scheme* con keccak256?

Es la garantía de inmutabilidad mediante Hash. En lugar de guardar los códigos secretos en texto plano dentro de la blockchain (donde cualquiera podría leerlos), el Notario guarda previamente la "huella digital" o Hash Criptográfico (keccak256) de cada código. Cuando el cliente introduce su código, el contrato calcula su hash y comprueba si coincide con la huella registrada.

2. Diseño

El contrato inteligente PromoEscrowLoyalty gestiona el ciclo de vida completo de la campaña promocional dividiéndolo en **4 fases secuenciales**:

Inicialización y Depósito de Escrow: Las marcas proveedoras registran la cantidad de recompensas y depositan los fondos en Wei/Ether en el contrato (depositEscrow).

Registro Notarial: El notario registra los hashes de los boletos ganadores. Este paso garantiza inmutabilidad antes de la distribución de boletos.

Distribución y Canje: El comercio entrega los boletos a los usuarios. Los usuarios ejecutan claimReward(string _code, bytes32 _salt) desde su *wallet*.

Verificación y Ejecución Automática:

- El contrato valida si el hash del código coincide con uno premiado.
- Comprueba que el código no haya sido reclamado previamente.
- Transfiere automáticamente la recompensa directamente desde el *Escrow* al usuario.

- Incrementa el contador de puntos del usuario y actualiza su nivel de fidelización.

2.1 Definición del Caso de Uso: Canjear Código

Nombre del Caso de Uso: Canjear Código Promocional y Cobro de Premio.

Actor Principal: Cliente / Usuario Final.

Precondiciones:

1. El Smart Contract ha sido desplegado por el Administrador (Supermercado).
2. La Marca Proveedora ha depositado fondos en la cuenta de custodia (*Escrow*) del contrato.
3. El Notario ha registrado el Hash criptográfico (keccak256) del código en el contrato.
4. El cliente posee una cartera criptográfica (MetaMask) conectada a la red.

Flujo Principal (Paso a Paso):

1. El cliente introduce su código promocional e invoca la función claimReward desde su *wallet*.
2. El Smart Contract calcula el Hash criptográfico del código introducido utilizando la función keccak256.
3. El contrato verifica si el Hash existe en el registro de códigos premiados.
4. El contrato comprueba que el código no haya sido reclamado previamente (isClaimed == false).
5. El contrato verifica si el cliente cumple con el nivel mínimo de fidelidad (*Tier*) requerido para el premio.
6. El contrato marca el código como usado (isClaimed = true).
7. El contrato transfiere automáticamente el dinero del premio desde el depósito en custodia (*Escrow*) de la marca hacia la *wallet* del cliente.
8. El contrato suma los puntos correspondientes al perfil del cliente y, si alcanza el umbral, incrementa su categoría de fidelidad (*Tier*).
9. El contrato emite un Evento público (RewardClaimed) para notificar la transacción a la aplicación Web3 / Frontend.

Postcondiciones:

El cliente recibe los fondos en su *wallet* de forma inmediata.

El código queda inhabilitado permanentemente para evitar doble cobro.

La categoría (*Tier*) y los puntos del cliente quedan actualizados en la blockchain.

2.2 Definición del Contenido del Smart Contract para el Caso de Uso "Canjear Código"

A. Datos y Estructuras de Datos

Estructura Premio:

uint256 amount: Monto en Wei/Ether asignado como premio.

address brand: Dirección de la marca proveedora que financia el premio.

bool isClaimed: Variable booleana que indica si el código ya fue canjeado.

uint8 tierRequired: Nivel o categoría mínima requerida del cliente (0, 1 o 2).

Estructura UserProfile (Perfil de Fidelización del Cliente):

uint32 totalPoints: Puntos acumulados por el cliente.

uint8 tier: Categoría actual de fidelidad del cliente (0 = Bronce, 1 = Plata, 2 = Oro).

Mapeos (Tablas de Almacenamiento):

mapping(bytes32 => Reward) public rewards: Relaciona el Hash criptográfico del código con la información del premio.

mapping(address => UserProfile) public users: Relaciona la dirección de la *wallet* del cliente con su perfil de puntos y categoría (*Tier*).

mapping(address => uint256) public brandEscrow: Almacena el saldo en custodia (*Escrow*) que ha depositado cada marca.

B. Funciones que Operan sobre los Datos:

claimReward(string calldata _code, bytes32 _salt) external:

Operación: Recibe el código en texto plano y una palabra secreta (*salt*), calcula su keccak256 y realiza las verificaciones de validación, transferencia de dinero, marcado de código usado y actualización de categoría (*Tier*) del cliente.

getUserProfile(address _user) external view returns (uint32 points, uint8 tier):

Operación: Función de sólo lectura (view) que permite al cliente consultar sus puntos acumulados y su categoría actual sin gastar gas.

C. Validaciones

El Hash generado con keccak256(_code, _salt) debe coincidir exactamente con una entrada existente en la tabla rewards.

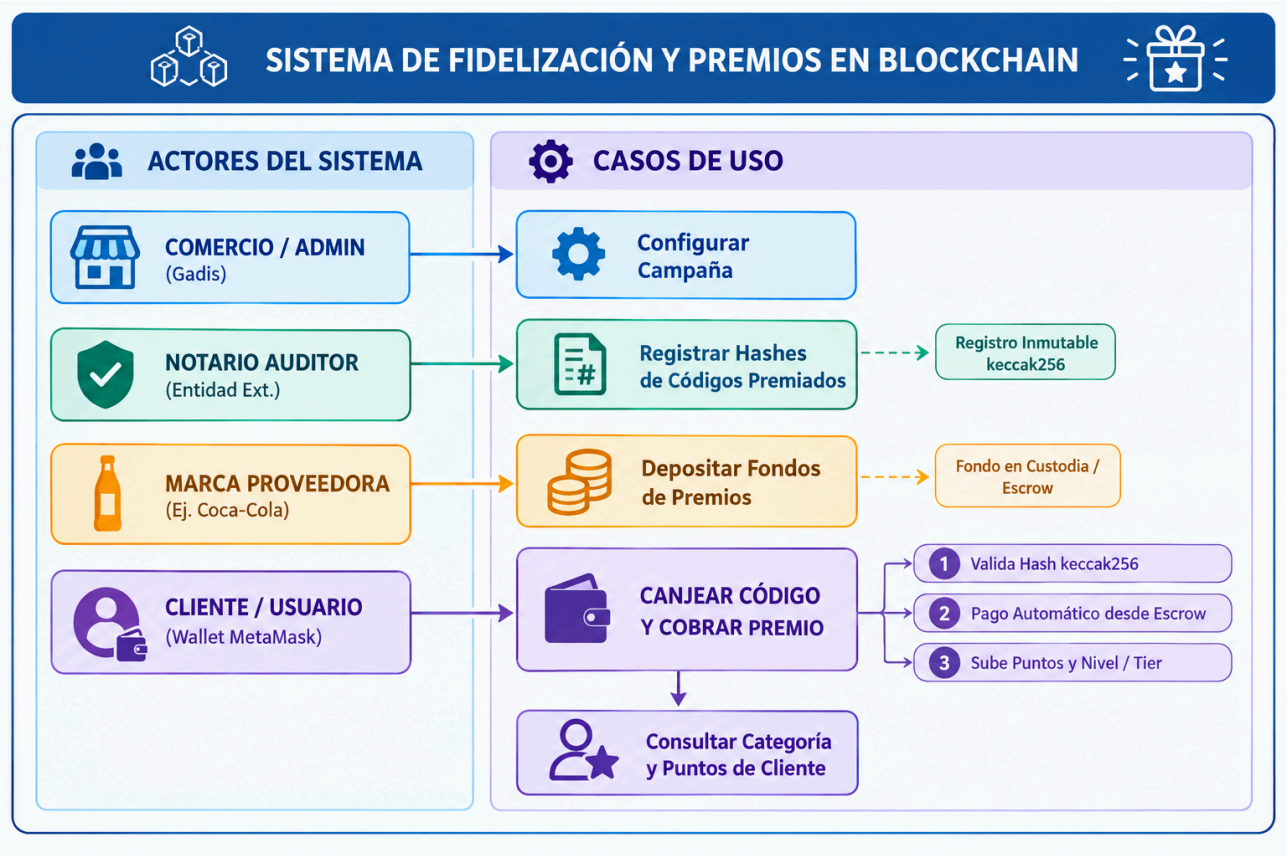
Anti-Doble Cobro: Si rewards[codeHash].isClaimed == true, la transacción se revierte con el mensaje de error *"El codigo ya ha sido canjeado"*.

Fondos Suficientes: El depósito en custodia de la marca debe ser mayor o igual al monto del premio (brandEscrow[brand] >= amount). En caso contrario, la transacción se revierte.

Categoría (*Tier*): El nivel del usuario debe ser mayor o igual al nivel exigido por el premio (users[msg.sender].tier >= tierRequired).

Seguridad de Transferencia: La actualización de estado (isClaimed = true) se ejecuta **antes** del envío del dinero para evitar ataques de reentrancia.

2.3 Usuarios/actores del sistema



3. Implementación

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.30;

/**
 * @title PromoEscrowLoyalty
 * @dev Enfoque exclusivo en el Caso de Uso: Canjear Código y Cobro de Premio.
 */
contract PromoEscrowLoyalty {

    // ========================================
    //  ESTRUCTURAS DE DATOS Y ESTADOS
    // ========================================

    struct Reward {
        uint256 amount;       // Monto del premio en Wei
        address brand;        // Marca proveedora que financia el premio
        bool isClaimed;       // Estado: true si ya fue canjeado
        uint8 tierRequired;   // Categoría mínima del cliente (0: Bronce, 1: Plata, 2: Oro)
    }

    struct UserProfile {
        uint32 totalPoints;   // Puntos acumulados de fidelidad
        uint8 tier;           // Categoría actual de fidelidad
    }

    // Mapeos de almacenamiento permanente
    mapping(bytes32 => Reward) public rewards;     // keccak256(codigo + salt) =>
Reward
    mapping(address => UserProfile) public users;   // Wallet cliente => Perfil de usuario

    // ========================================
    //  EVENTOS
    // ========================================

    event RewardClaimed(address indexed winner, bytes32 indexed codeHash, uint256
amount, uint8 newTier);

    // ========================================
    //  CONSTRUCTOR
    // ========================================

    constructor() payable {
        // Se puede inicializar con fondos ETH para respaldar los premios del caso de uso
    }
```

```
    // ========================================
    //  CASO DE USO PRINCIPAL: CANJEAR CÓDIGO
    // ========================================

    /**
     * @dev Permite a un Cliente introducir un código promocional, validar el Hash keccak256,
     *      recibir el premio y actualizar sus puntos/tier.
     * @param _code Código promocional en texto plano.
     * @param _salt Frase/palabra secreta asociada al código.
     */
    function claimReward(string calldata _code, bytes32 _salt) external {
        // 1. Reconstrucción del Hash criptográfico
        bytes32 codeHash = keccak256(abi.encodePacked(_code, _salt));
        Reward storage reward = rewards[codeHash];

        // 2. Validaciones (Checks)
        require(reward.amount > 0, "El codigo no existe o no esta premiado");
        require(!reward.isClaimed, "El codigo ya ha sido canjeado");

        UserProfile storage user = users[msg.sender];
        require(user.tier >= reward.tierRequired, "Nivel (Tier) de usuario insuficiente");

        // 3. Modificaciones de Estado (Effects) - Patrón Anti-Reentrancia
        reward.isClaimed = true;

        // Sumar puntos de fidelidad
        user.totalPoints += 100;

        // Promoción de Tier basada en puntos acumulados
        if (user.totalPoints >= 500 && user.tier < 2) {
            user.tier = 2; // Sube a nivel ORO
        } else if (user.totalPoints >= 200 && user.tier < 1) {
            user.tier = 1; // Sube a nivel PLATA
        }

        uint256 claimAmount = reward.amount;

        emit RewardClaimed(msg.sender, codeHash, claimAmount, user.tier);

        // 4. Interacción Externa / Pago (Interactions)
        (bool success, ) = payable(msg.sender).call{value: claimAmount}("");
        require(success, "Fallo al transferir el premio en ETH");
    }
    // ========================================
    //  Funciones Auxiliares de Consulta
    // ========================================

    /**
     * @dev genera el hash
```

```
    */
    function getPromoCodeHash(string calldata _code,string calldata _salt) external pure
returns(bytes32) {
        return keccak256(abi.encodePacked(_code,_salt));
    }

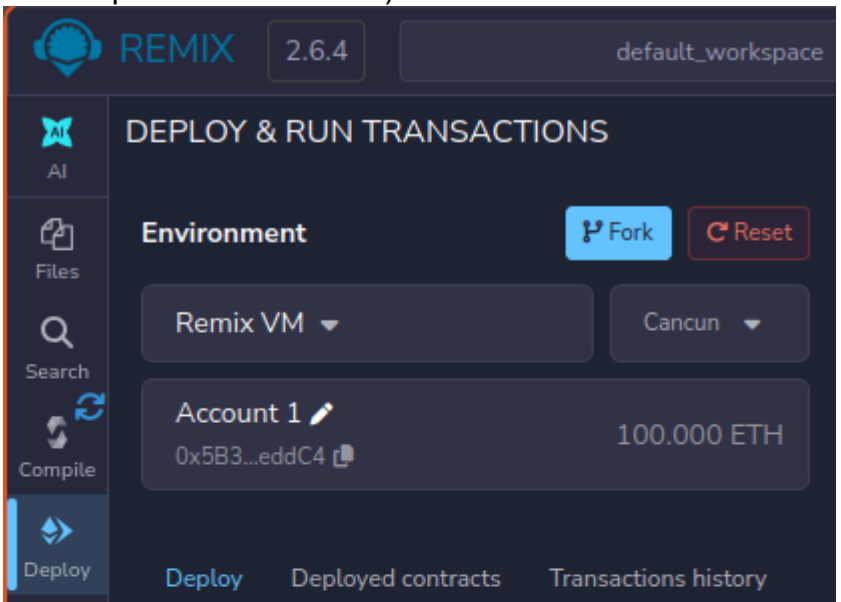
    /**
     * @dev Consulta la información del perfil del cliente (Solo lectura, no gasta gas).
     */
    function getUserProfile(address _user) external view returns (uint32 points, uint8 tier) {
        UserProfile memory u = users[_user];
        return (u.totalPoints, u.tier);
    }
}
```

ya que el código no debe tener vulnerabilidades y debemos optimizar el gas:

Se marca “reward.isClaimed = true” y se actualizan los puntos antes de enviar los fondos, evitando cualquier vulnerabilidad de reentrancia.
Los parámetros de entrada como cadenas de texto y arrays utilizan “calldata” en lugar de “memory”, reduciendo significativamente el consumo de gas durante la llamada a las funciones.
En UserProfile, uint32 y uint8 están empaquetados en un mismo *slot* de almacenamiento de 256 bits, reduciendo los costes en gas de escritura

4. Pruebas

Ponemos en el campo VALUE 10 y seleccionamos la unidad Ether (esto simula cargar el contrato con fondos de premios en *Escrow*).



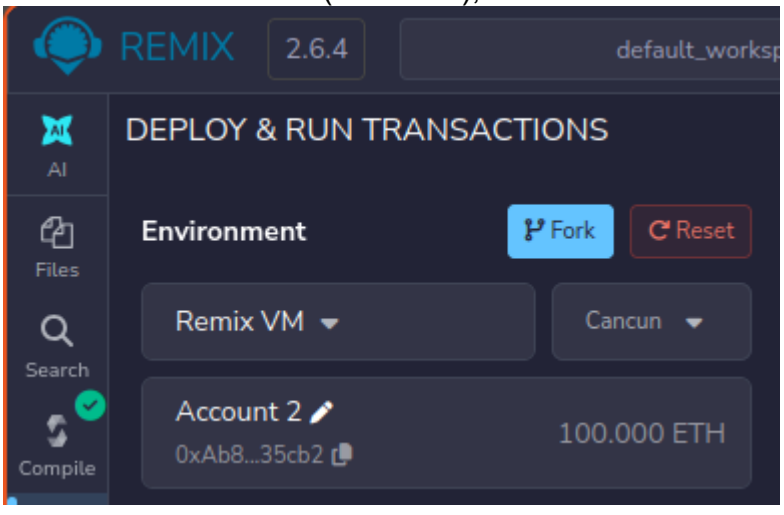
Compilamos y hacemos click en DEPLOY

En el momento de darle al botón Deploy, le estamos transfiriendo **10 ETH ficticios** de la cuenta de pruebas de Remix al saldo interno del contrato, de esta forma, el contrato pasa a custodiar esos 10 ETH en su propia cuenta (lo que llamamos la *cuenta de custodia* o *Escrow*). Cuando el usuario ejecuta el canje, el contrato puede transferirle el premio directamente desde ese fondo acumulado.

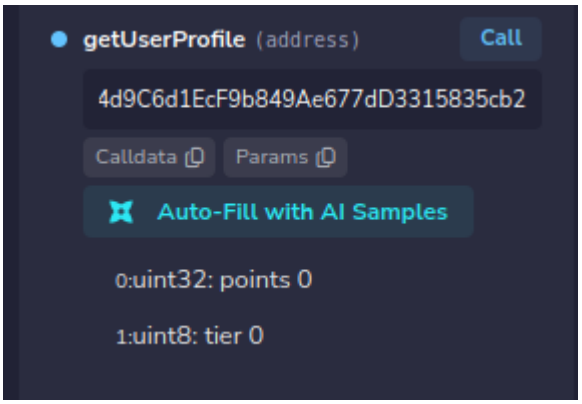
Prueba 1: Consulta del Perfil Inicial del Cliente (Sin Puntos)

Objetivo: Verificar que un cliente nuevo inicia con 0 puntos y categoría (*Tier*) 0 (Bronce)

En el desplegable de cuentas de Remix (**Account**), seleccionamos la **Cuenta #2**



En la función getUserProfile, introducimos la dirección de la **Cuenta #2** y hacemos clic en **call**[



Prueba 2: Inserción de Código Premiado

Objetivo: Simular un código registrado previamente en el mapa rewards para poder probar el canje.

Dado que en el código no pusimos la función de notario para registrar boletos, la forma más sencilla de simular la Prueba 2 (Registro del Hash Premiado) es incluir un código de prueba en el constructor del contrato.

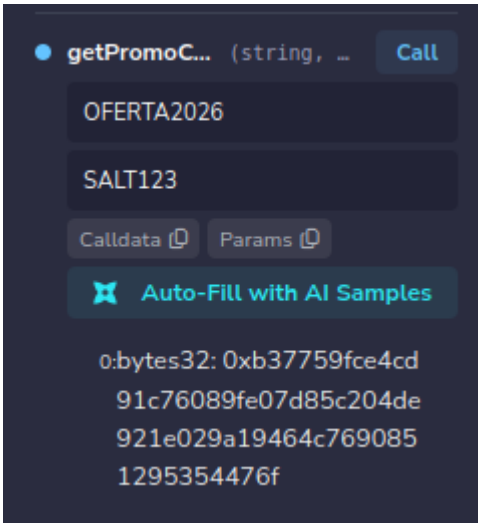
```
constructor() payable {
    // Definimos el código y el salt exactamente igual que como los pasará el usuario
    string memory code = "OFERTA2026";
    string memory salt = "SALT123";

    // Generamos el hash con abi.encodePacked igual que en claimReward
    bytes32 promoHash = keccak256(abi.encodePacked(code, salt));

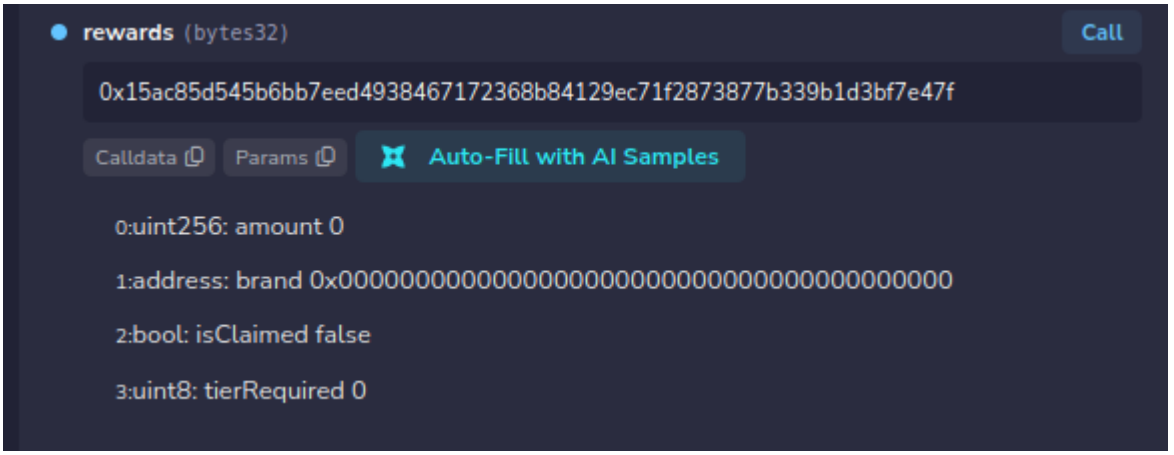
    // Guardamos la recompensa de 1 ETH
    rewards[promoHash] = Reward({
        amount: 1 ether,
        brand: msg.sender,
        isClaimed: false,
```

```
        tierRequired: 0
    });
}
```

Calculamos el hash premiados



Introducimos un hash no premiado



Introducimos el hash premiado.

0:



uint256: amount 1000000000000000000 Equivale exactamente a 1 ETH.

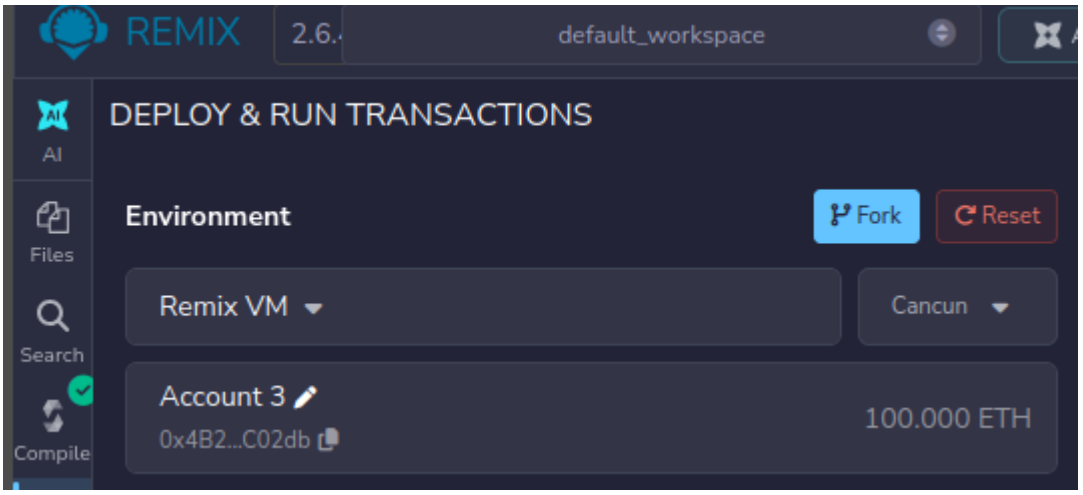
1: address: brand 0xAb84... Muestra la dirección del creador/marca.

2: bool: isClaimed false Indica que el premio aún no ha sido canjeado.

Prueba 3: Canje Exitoso de Código, Cobro de ETH y Aumento de Puntos

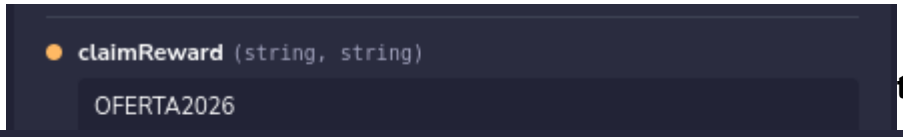
Objetivo: Validar que el cliente que ejecuta claimReward, recibe los ETH del premio y suma puntos/tier

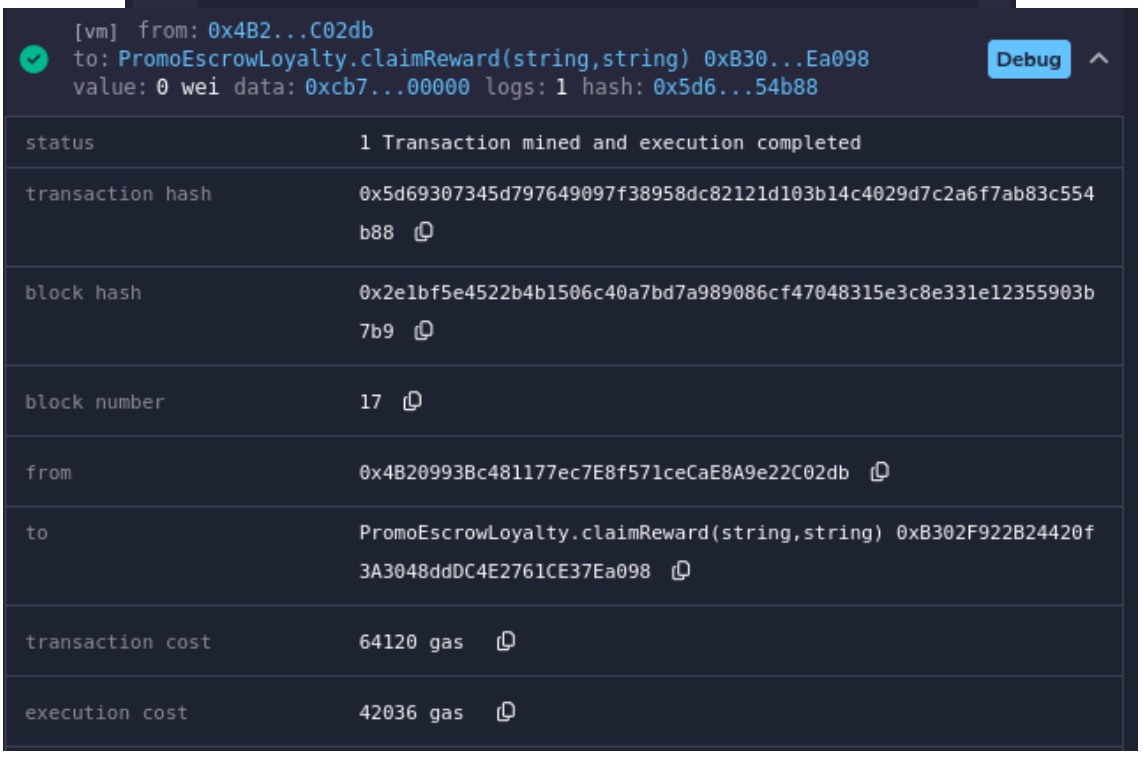
Seleccionada la Cuenta 3



Introducimos los datos premiados

Pulsamos en **transact**

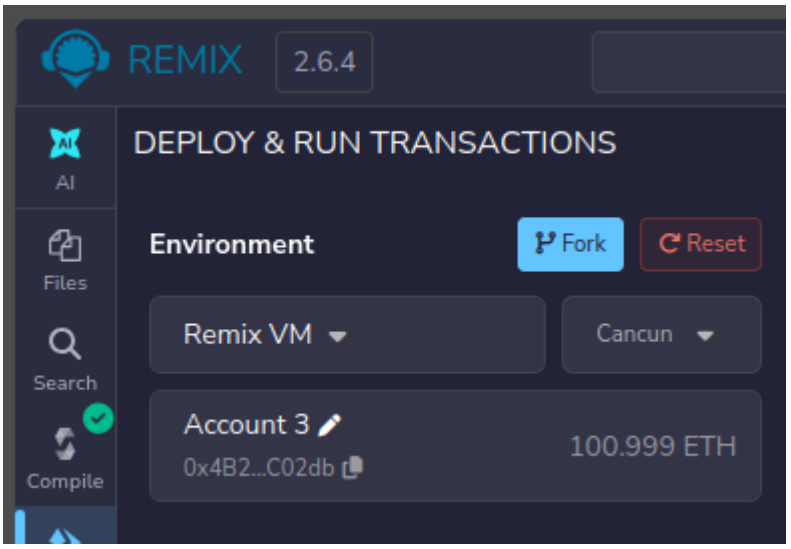




Resultado Esperado:

Transacción confirmada en verde en la consola de Remix.

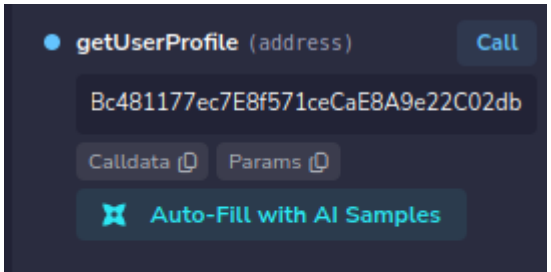
El saldo de la **Cuenta 33** se incrementa a ~101 ETH (menos el pequeño coste de gas).



Se emite el evento RewardClaimed con la dirección del ganador y el importe.



Al consultar getUserProfile, los puntos han subido a 100.



Prueba 4: Verificación de Regla Anti-Doble Cobro (Ataque/Fallido)

Objetivo: Comprobar que la regla de seguridad del contrato impide canjear dos veces el mismo código.

Sin cambiar de cuenta, vollvemos a pulsar el botón transact en la función claimReward utilizando exactamente los mismos parámetros anteriores ("OFERTA2026", _salt).



La transacción se **revierte en rojo**.

La consola de Remix muestra el mensaje de error de nuestra validación:

Reason provided by the contract: "El codigo ya ha sido canjeado".

El saldo en ETH del cliente no cambia y el estado del contrato se mantiene íntegro.

Comprobamos que el código ya se ha usado:


rewards (bytes32)
Call
0xb37759fce4cd91c76089fe07d85c204de921e029a19464c7690851295354476f
Calldata
Params
Auto-Fill with AI Samples
0:uint256: amount 1000000000000000000
1:address: brand 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
2:bool: isClaimed true
3:uint8: tierRequired 0